End-to-End Execution Trace

Documented checkout: `4405d167fc323f33be76e2c7f6614fd21cc574a8` on `main`. This volume links previously explained functions into full journeys and names what runs once at startup, per HTTP request, and offline. Values in the worked walk-through are labeled illustrative unless directly fixed by code/data.

1. Three execution timelines

A. Offline data/model build

1. `pipeline/01_audit_and_clean_data.py` reads `data/raw/DELHI_METRO_DATA.csv`, cleans/normalizes station rows, applies hand-authored corrections and writes `data/cleaned/stations_cleaned.csv`, compatibility copies and operations settings. This step does **not** establish external provenance for each manually authored coordinate/operating assumption. 2. `pipeline/02_generate_calibrated_dataset.py` reads cleaned stations, writes typology and interchange split JSON, generates two years of synthetic station/hour rows using a fixed random seed, and writes large CSVs. Those two CSV paths are ignored by Git. The generator uses stochastic Poisson sampling around configured expectations; reproducible seed gives repeatability for same software/input order, not evidence of observed ridership. 3. `pipeline/03_train_and_evaluate_forecaster.py` reads the synthetic CSV and metadata, prepares temporal/calendar/station features, separates a date holdout at `SPLIT_DATE`, trains the configured estimator and Ridge baseline, computes metrics, and serializes model/metadata/metrics files. 4. Runtime `app/forecasting.py` loads the saved model and metadata. Production inference does not automatically rerun pipelines or retrain. Changing the generated data alone does not alter predictions until the model is retrained and serialized artifacts are replaced.

Data contract chain: raw station directory -> cleaned station directory -> generated scenario records -> training feature matrix/target -> sklearn model + metadata -> runtime model-ready row -> predicted demand.

2. Startup and import-time state

1. Render executes the `Procfile` command `gunicorn run:app`; Gunicorn imports `run.py` and finds `app`. 2. `run.py` imports `app` package. `app/__init__.py` constructs Flask and imports `app.routes` so decorators register endpoints. 3. `routes.py` reads cleaned station data (with a fallback path), derives station lists/defaults, imports forecasting and scheduling helpers, and builds the travel graph from station CSV plus topology. 4. Importing forecasting loads `.joblib` model/metadata and station CSV, and optionally evaluation metrics. Importing the optimizer loads operations CSV and line-share JSON.

Consequences: one missing/corrupt required file or incompatible serialized estimator may fail worker import before serving requests. Multiple workers repeat their own in-memory loads. A local virtualenv is excluded from deployment; requirements must install compatible dependencies. There is no database connection/persistent store in the tracked app path.

3. `/api/predict` request walkthrough

Browser sends

`dashboard.js:fetchPrediction` reads `station`, `date`, `time`, and selected `line` controls and sends a JSON POST. The backend also accepts query args as fallback when parsed JSON is empty. The browser assigns each request an increasing ID; an older response is ignored if a newer request exists.

Route parsing and context

`routes.api_predict` defaults station to Rajiv Chowk and time to 09:00, trims station and removes footnote glyphs, then calls `_parse_datetime`. Only `ValueError` is caught here and returned as JSON status error with HTTP 400. It computes calendar context and asks `is_service_operating(hour, minute)`. It obtains station metadata before prediction.

Prediction branch

If service is open, it calls minute interpolation and hourly prediction. If closed, both selected-time demand values become zero without invoking those two functions. It always builds `profile_24h`; that profile iterates operating hours 05 through 23. The minute interpolation blends current-hour and next-hour predictions linearly according to `minute / 60`; the model only predicts hourly anchors.

Schedule branch

The route calls `optimize_station_schedule` with station, primary line, all lines, hour, Python weekday index, hourly footfall, calendar workday flag, and service flag. For interchange station with no line filter, it constructs a station-wide headline by summing each line's boarding-clearance capacity and train counts, then calculates combined utilization. A chosen line uses that line's recommendation. If a nonempty line filter does not match any line, the current code silently leaves the previous default recommendation selected; it does not return a validation error.

Closed-service processing overrides the headline to zero demand, no headway, no capacity, N/A utilization/wait, no trains, and a closed badge. The per-line recommendation list is also produced by the optimizer with closed status. This is a display/model gate, not proof no passenger can be on trains at that instant; the app treats all modeled demand as zero outside its configured operating window.

Routing and response

The route runs multi-source Dijkstra from every line-node of the origin and converts the station-keyed result to a sorted array of destination, rounded minutes and one line label from the station CSV's primary `Line` column. The travel time is a topology-weight estimate, not a departure-time timetable prediction. The `station_line_map` does not necessarily represent every destination interchange line.

It appends selected hour-band/peak context and returns JSON fields: status, station, station_info, datetime, calendar_context, footfall, scheduling (headline/all lines/interchange flag), profile_24h, travel_times, and explicit `data_provenance` disclaimers. Browser renderers map these values into visible text, chart series, doughnut breakdown and filterable destinations.

4. Worked request example (illustrative)

Request: `POST /api/predict` body `{station: "Rajiv Chowk", date: "2026-10-14", time: "09:00", line: ""}`. This demonstrates control/data flow; predicted numeric ridership is not claimed here.

1. Browser request ID increments; JSON request goes to Flask. 2. `_parse_datetime` creates a Python datetime, hour 9 and minute 0. Calendar context marks date facts based on current static calendar code. Operating gate evaluates 09:00 as open. 3. `get_station_info` returns interchange metadata and both line names. 4. Forecasting produces hourly and minute values, plus a 19-point 05:00-23:00 profile. The profile's maximum index determines peak hour. 5. Scheduling distributes station hourly demand according to stored share proportions (or equal fallback), then runs one line schedule per line. Since line filter is empty and this is an interchange, API creates a combined headline. `recommended_clearance_capacity` is sum of line-level hourly boarding clearance; utilization is station demand divided by this sum. 6. Dijkstra starts from Rajiv Chowk's Blue and Yellow nodes, explores adjacent line edges and interchange links, then keeps minimum arrival value per station. 7. JSON response arrives. If this is still latest request ID, frontend sets text values, selects line/all-line curve as applicable, draws Chart.js charts, fills line table and destination list.

To debug a mismatch, compare in this order: request payload -> route JSON -> `scheduling.headline` -> DOM field. That locates whether an error originates in control state, backend calculation, serialization, or rendering.

5. Error paths and reliability boundaries

Failure/input	Current observed behavior	Practical effect
Bad date/time parse	Route returns JSON error + 400	Browser shows server message in status element
Unknown station	`get_station_info` returns unknown metadata; graph route can yield empty travel result; forecaster uses index fallback for feature encoding	Demand may look like another station's encoded baseline; station selection is not explicitly rejected in this API path
JSON parse/network failure	Browser catch logs and shows generic service error	Existing dashboard content can remain visible/stale
HTTP non-2xx with valid JSON	Browser does not inspect `res.ok`; examines JSON status	Usually shows API message if error payload follows contract
HTTP error with HTML body	`res.json()` throws; generic catch path	Backend HTML error details are not shown to user
Missing startup artifact	Import-time file/model load can fail	Worker may not boot; no runtime recovery path shown
Graph edge mismatch/disconnected nodes	Dijkstra returns only reachable graph destinations, or `{}` when no source node	UI count can be lower than station inventory; no route-level warning is added
Empty station `all_lines`	Scheduler later does `max` on empty recommendation list	Request may fail unless upstream station metadata prevents it

The API lacks an explicit schema validator for station/time/line values. There is no automated test suite tracked. These are verified source observations; proposed mitigations are not implemented in this revision.

6. Security and data-quality crosscutting notes

• Dashboard renderers use `textContent` for many text fields, but use `innerHTML` for dynamic line rows and destination rows; treat those as injection sinks if upstream text becomes untrusted.

• `tojson` protects template embedding for station metadata and landing profiles; avoid replacing it with string interpolation.

• `.joblib` is a Python pickle-like trusted artifact format; do not load files from an untrusted upload/source.

• CSV values are parsed as numbers but operation assumption validation is not comprehensive.

• Synthetic model holdout evaluation checks prediction consistency against synthetic generated targets, not real measured DMRC footfall. Explain this distinction in interviews.

• The deployment ignores generated 130 MB CSVs. Current serving loads serialized model/metadata and smaller runtime CSVs; retraining pipeline needs generated CSV or a regeneration step. Deployment build does not run the training pipeline automatically.

7. Interview narrative with technical precision

"MetroInsights is a Flask application that serves station-level demand estimates from a scikit-learn model trained on generated synthetic scenarios. A browser posts station/date/time/line selections to an API. The backend applies date and service-window context, runs hourly inference and minute interpolation, distributes interchange demand across lines, and produces a rule-based frequency/capacity scenario. A separate topology-aware weighted graph estimates shortest station-to-station travel times. The frontend updates KPIs, charts and tables without a full-page reload. Its principal limits are synthetic rather than observed training targets, configurable rather than verified operating assumptions, simplified boarding-capacity calculations, and estimated rather than timetable-derived travel time."

The statement is intentionally explicit about what is and is not live. Avoid calling the service "real-time ridership prediction" unless the data source and deployment are changed to support that claim.

8. Integrated exercises

1. Trace a closed-hour request and list which forecast calls are skipped, which profile calls still occur, and which response keys represent zero or `None`. 2. Alter a station line dropdown in DevTools and predict how demand, chart historical baseline, headline utilization and recommendation fields change. 3. Explain why a destination time can be present without a direct track edge: Dijkstra may pass through multiple line nodes and transfer edges. 4. Propose a contract test that checks JSON keys and a frontend test that verifies `dispUtilization` reflects the headline. 5. Audit a generated response's provenance labels and state exactly which claims are synthetic, configurable or project route-map data.

Solutions: (1) `service_open` false makes selected minute/hourly demand zero, but full-day profile still makes hourly model calls for 05-23; optimizer still constructs closed line recommendation records, then the API overrides headline; keys include `service_open:false`, demand 0, capacity 0, utilization/wait `None`, headways `None`. (2) Only if a station supports the selected line does the option appear. A selected line sets headline to that line, uses line curve, and scales historical values using the selected-line/current total share; schedule values are line-level. (3) The graph's path is a sequence of weighted adjacent track edges plus transfer penalties. (4) Assert status success and expected response schema with fixture inputs; in browser, mock JSON and check text content. (5) Read `data_provenance` and do not elevate those labels to external verification.

Appendix A - complete source excerpts

These excerpts come from the documented commit. Original line numbers are printed at left. For print readability, Unicode dash variants are normalized to ASCII hyphens and other non-ASCII source symbols are shown as explicit Unicode escape notation; this does not change the source line numbers or code meaning. Long lines wrap with indented continuations.

Complete end-to-end prediction API route (api_predict) `app/routes.py:234-398`

```
234 @app.route('/api/predict', methods=['GET', 'POST'])
235 def api_predict():
236     """
237     JSON API for dynamic frontend interaction without full page reloads.
238     """
239     data = request.get_json(silent=True) or request.args
240     station = data.get('station', 'Rajiv Chowk').strip().rstrip('\u2020\u2021*#\u00a4').strip()
241     date_str = data.get('date', '')
242     time_str = data.get('time', '09:00')
243     line_filter = data.get('line', '')
244 
245     try:
246         dt_obj, hour, minute = _parse_datetime(date_str, time_str)
247     except ValueError as exc:
248         return jsonify({"status": "error", "message": str(exc)}), 400
249     calendar_context = get_calendar_context(dt_obj.date())
250     service_open = is_service_operating(hour, minute)
251     calendar_context['service_open'] = service_open
252     st_info = get_station_info(station)
253 
254     # 1. Predictions
255     time_footfall = predict_footfall_with_minute_interpolation(station, dt_obj, hour, minute) if service_open else 0
256     hourly_footfall = predict_hourly_footfall(station, dt_obj, hour) if service_open else 0
257     profile_24h = predict_station_full_day_profile(station, dt_obj)
258 
259     # 2. Scheduling Optimization
260     schedule_res = optimize_station_schedule(
261         station=station,
262         primary_line=st_info['line'],
263         all_lines=st_info['all_lines'],
264         hour=hour,
265         day_of_week=dt_obj.weekday(),
266         total_footfall=hourly_footfall,
267         is_workday=calendar_context['is_workday'],
268         service_open=service_open
269     )
270 
271     # Filter if requested
272     filtered_rec = schedule_res['primary_recommendation']
273     station_summary = not line_filter and schedule_res['is_interchange']
274     if line_filter:
275         for r in schedule_res['line_recommendations']:
276             if r['line'].lower() == line_filter.lower():
277                 filtered_rec = r
278                 break
279     elif station_summary:
280         # The demand chart/KPI describe the whole interchange. Aggregate service
281         # across every corridor too, so the headline never compares station-wide
282         # passengers with just one line's trains or capacity.
283         recommendations = schedule_res['line_recommendations']
284         total_demand = schedule_res['total_station_footfall']
285         recommended_capacity = sum(r['recommended_clearance_capacity'] for r in recommendations)
286         existing_capacity = sum(r['existing_clearance_capacity'] for r in recommendations)
287         utilization = round(total_demand / recommended_capacity * 100, 1) if recommended_capacity else 0.0
288         if utilization > 105.0:
289             risk, badge, color = 'CRITICAL_OVERCROWDING', 'Severe Crowd Alert', '#dc3545'
290         elif utilization > 88.0:
291             risk, badge, color = 'HIGH_CROWDING', 'High Demand', '#fd7e14'
292         elif utilization > 65.0:
293             risk, badge, color = 'MODERATE', 'Optimal Load', '#28a745'
294         else:
295             risk, badge, color = 'LOW', 'Comfortable Capacity', '#17a2b8'
296         total_trains = lambda key: round(sum(r[key] for r in recommendations), 1)
297         total_fleet = sum(r['fleet_size_available'] for r in recommendations)
298         demand_weight = max(total_demand, 1)
299         filtered_rec = {
300             **schedule_res['primary_recommendation'],
301             'line': 'All lines',
302             'line_demand': total_demand,
303             'is_station_summary': True,
304             'line_count': len(recommendations),
305             'recommended_headway_min': None,
306             'existing_headway_min': None,
307             'recommended_trains_per_hour': total_trains('recommended_trains_per_hour'),
308             'existing_trains_per_hour': total_trains('existing_trains_per_hour'),
309             'recommended_clearance_capacity': recommended_capacity,
310             'existing_clearance_capacity': existing_capacity,
311             'capacity_utilization_pct': utilization,
312             'crowding_risk': risk,
313             'crowding_badge': badge,
314             'crowding_color': color,
315             'avg_wait_time_rec_min': round(sum(r['avg_wait_time_rec_min'] * r['line_demand'] for r in recommendations) /
    demand_weight, 1),
316             'trains_deployed_rec': sum(r['trains_deployed_rec'] for r in recommendations),
317             'fleet_size_available': total_fleet,
318             'explanation': (
319                 f"Combined estimate across {len(recommendations)} lines: {total_demand:,} passengers/hr. "
320                 f"The recommended schedules provide an estimated comfortable boarding capacity of "
321                 f"{recommended_capacity:,} passengers/hr ({utilization}% utilization). "
322                 "This is a model estimate based on configurable train-capacity assumptions, not a live passenger count."
323             )
324         }
325 
326     if not service_open:
327         filtered_rec = {
328             **filtered_rec,
329             'line': 'All lines' if station_summary else filtered_rec['line'],
330             'line_demand': 0,
331             'recommended_headway_min': None,
332             'existing_headway_min': None,
333             'recommended_trains_per_hour': 0,
334             'existing_trains_per_hour': 0,
335             'recommended_clearance_capacity': 0,
336             'existing_clearance_capacity': 0,
337             'capacity_utilization_pct': None,
338             'crowding_risk': 'SERVICE_CLOSED',
339             'crowding_badge': 'Metro closed',
340             'crowding_color': '#64748b',
341             'avg_wait_time_rec_min': None,
342             'trains_deployed_rec': 0,
343             'status_action': 'SERVICE_CLOSED',
344             'explanation': f"Metro service is closed at {hour:02d}:{minute:02d} (modeled operating window: 05:00-23:30). No trains
     are operating and modeled passenger demand is zero."
345         }
346 
347     # 3. Dijkstra Travel Times to ALL stations
```

```
    times = shortest_times_from(travel_graph, station)
    station_line_map = dict(zip(stations_df['Station'], stations_df['Line']))
    travel_list = [
        {
            'destination': dest,
            'travel_time_min': round(mins, 1),
            'line': station_line_map.get(dest, 'Metro Network')
        }
        for dest, mins in sorted(times.items(), key=lambda kv: kv[1])
    ]

    calendar_context['hour_band'] = _hour_band(profile_24h['predicted'], hour, minute)
    calendar_context['selected_hour_demand'] = hourly_footfall
    calendar_context['daily_peak_hour'] = profile_24h['peak_hour']

    return jsonify({
        "status": "success",
        "station": station,
        "station_info": st_info,
        "datetime": {
            "date": dt_obj.strftime('%Y-%m-%d'),
            "day": dt_obj.strftime('%A'),
            "time": f"{hour:02d}:{minute:02d}",
            "hour": hour,
            "minute": minute,
            "is_peak": filtered_rec['is_peak'],
            "service_open": service_open
        },
        "calendar_context": calendar_context,
        "footfall": {
            "instant_interpolated": time_footfall,
            "hourly_demand": hourly_footfall,
            "daily_total": profile_24h['daily_total'],
            "station_peak_hour": profile_24h['peak_hour'],
            "station_peak_footfall": profile_24h['peak_footfall'],
        },
        "scheduling": {
            "headline": filtered_rec,
            "all_lines": schedule_res['line_recommendations'],
            "is_interchange": schedule_res['is_interchange']
        },
        "profile_24h": profile_24h,
        "travel_times": travel_list,
        "data_provenance": {
            "station_topology": "PROJECT_ROUTE_MAP_DATA_REQUIRES_SOURCE_VERIFICATION",
            "train_capacities": "CONFIGURABLE_PROJECT_ASSUMPTIONS_NOT_VERIFIED_DMRC_SPECIFICATIONS",
            "baseline_headways": "CONFIGURABLE_PROJECT_ASSUMPTIONS_NOT_VERIFIED_DMRC_TIMETABLES",
            "hourly_footfall_forecast": "MODEL_ESTIMATE_TRAINED_ON_SYNTHETIC_SIMULATION_NOT_OBSERVED_COUNTS",
            "interchange_demand_distribution": "SYNTHETIC_WEIGHTED_ESTIMATE_NOT_OBSERVED_LINE_SPLIT"
        }
    })
```